

高性能 Matrix 算子开发

结合 Ascend C 开源手册与样例

从入门到高阶，理解优化原理与芯片架构

作者：郑朝

时间：2026年7月

目录

Part 1 昇腾矩阵计算基础

Part 2 矩阵算子编程实践

2.1 基础编程入门

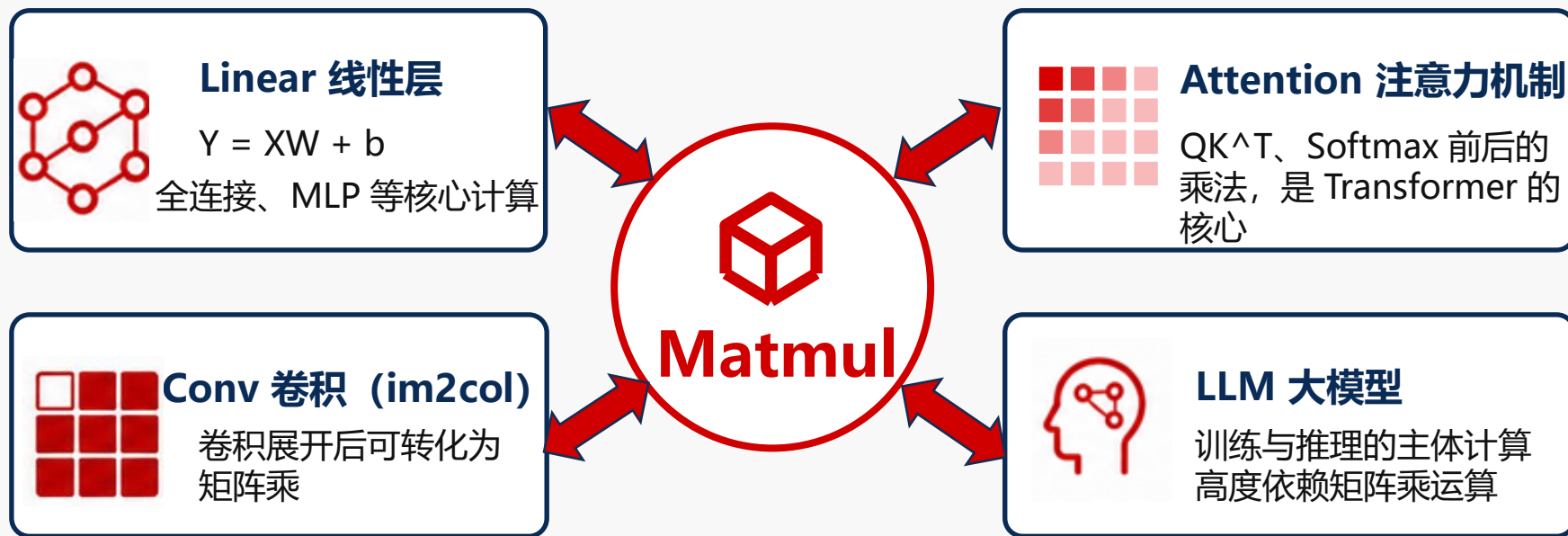
2.2 核心特性学习

2.3 高性能开发实践

Part 3 能力全景与开发路径

01 昇腾矩阵计算基础：矩阵运算的应用场景

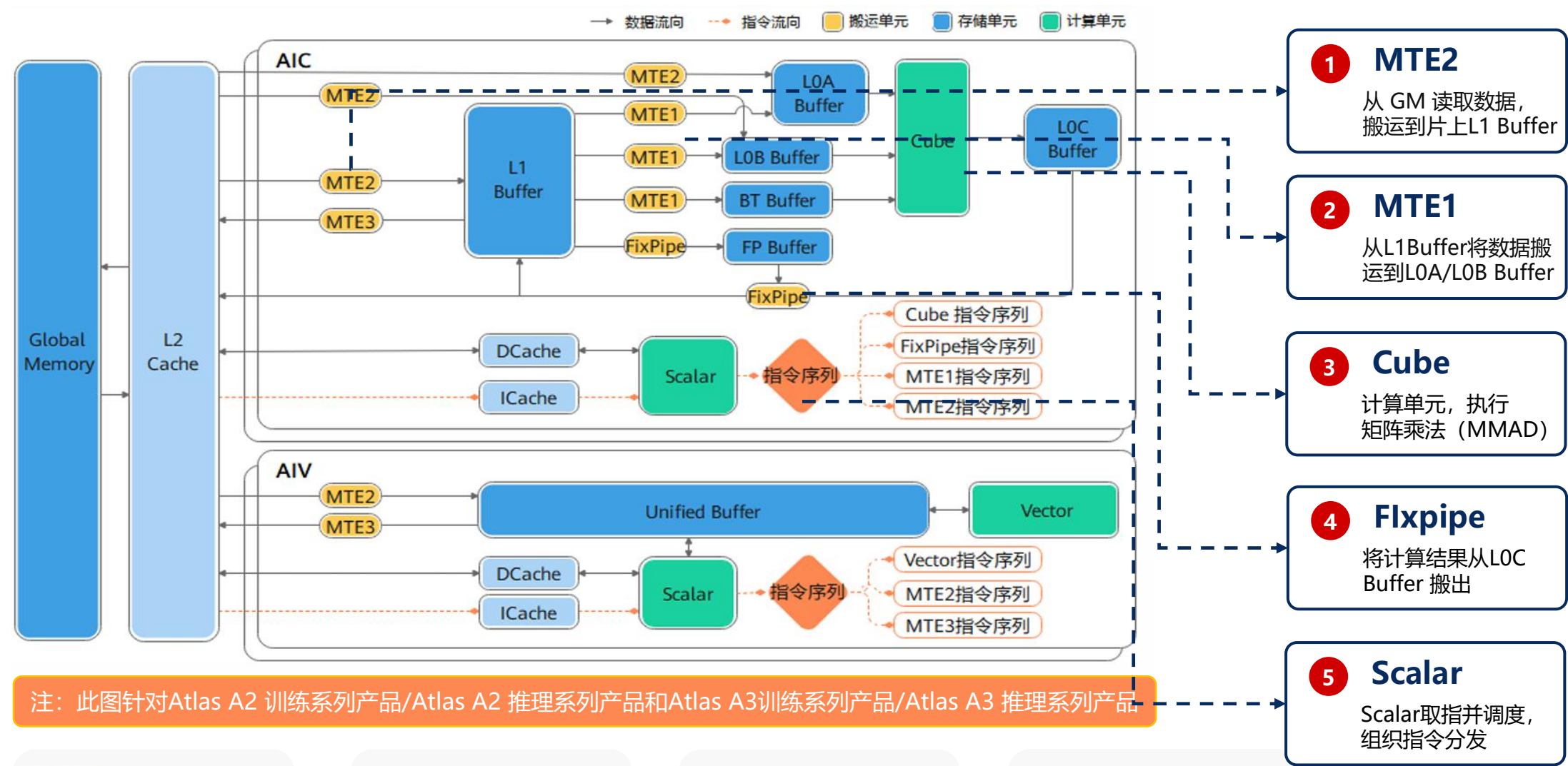
- 矩阵乘是**深度学习计算基石**：线性层、注意力机制、卷积 im2col 后都可归约为矩阵乘
- 大模型训练与推理中，Matmul 及 BatchMatmul、GroupedMatmul 通常**占据主要算力开销**



调优方法可迁移到 Conv、FlashAttention 等 Cube 类算子

CANN

01 昇腾矩阵计算基础：芯片架构与Cube计算数据流



Cube 单元：
矩阵乘累加

片上缓存：
L1/L0A/L0B/L0C
Buffer

Scalar：
取指与调度

数据流：GM->L1->L0A/B
Cube->L0C->GM

目录

Part 1 昇腾矩阵计算基础

Part 2 矩阵算子编程实践

2.1 基础编程入门

2.2 核心特性学习

2.3 高性能开发实践

Part 3 能力全景与开发路径

02 矩阵算子编程实践：总体学习路径

同一套 Ascend C 矢量编程范式，逐级提升性能与工程能力

3 高阶

高阶：矩阵算子性能阶梯优化

Case 0-8
性能阶梯优化

理论性能计算

映射到样例与手册

样例目录：

https://gitcode.com/cann/asc-devkit/tree/master/examples/01_simd_cpp_api/05_best_practices/01_matrix_compute

手册章节

<https://asc.gitcode.com/guide/算子实践参考/SIMD算子性能优化/SIMD算子性能优化.html>

2 中阶

中阶：数据通路与同步

GM-> L1 Buffer
Datacopy

L1 Buffer->L0A/B
LoadData

Cube 计算
Mmad

L0C Buffer -> GM
Fixpipe

流水同步
Set/Wait

映射到样例与手册

样例目录：

https://gitcode.com/cann/asc-devkit/tree/master/examples/01_simd_cpp_api/03_basic_api

手册章节

<https://asc.gitcode.com/SIMD-API/基础API/矩阵计算（ISASI）/矩阵计算（ISASI）.html>

1 入门

入门：Introduction / Basic API

跑通 Matrix
端到端链路

熟悉 Ascend C
编程模型与接口

数据流
四大通路

正确性验证
对齐 Host 结果

映射到样例与手册

样例目录

https://gitcode.com/cann/asc-devkit/blob/master/examples/01_simd_cpp_api/00_introduction

手册章节

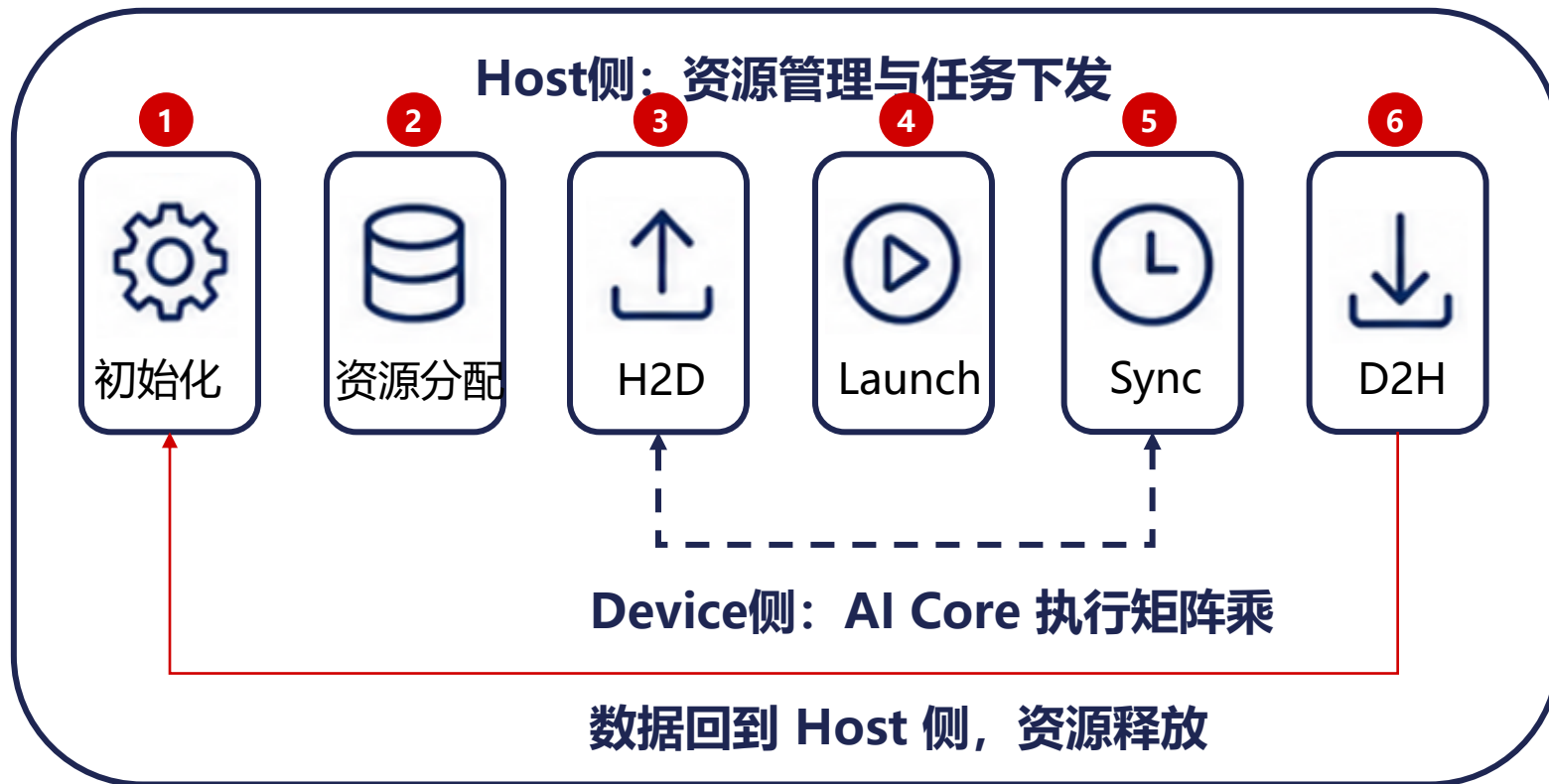
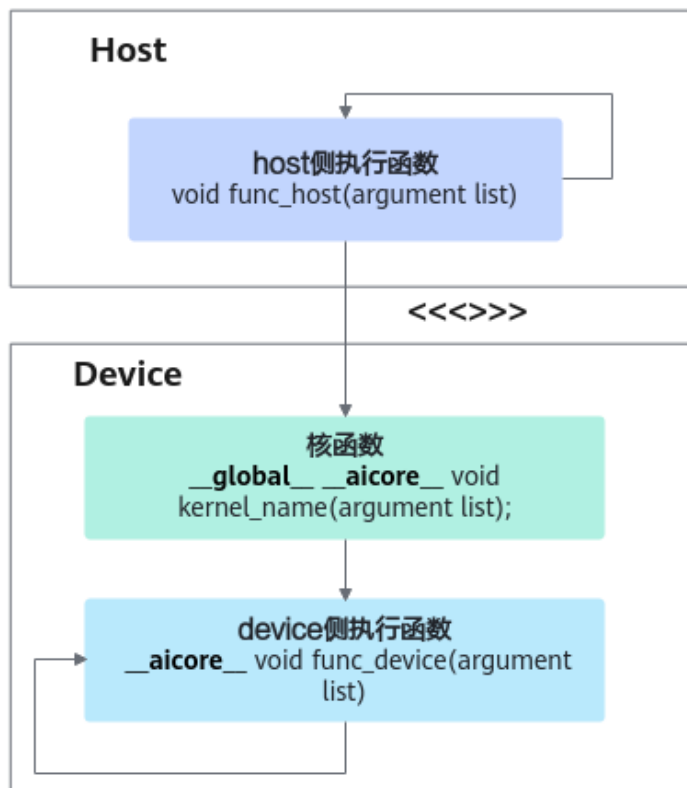
<https://asc.gitcode.com/SIMD-API/基础API/矩阵计算（ISASI）/矩阵计算（ISASI）.html>

成长路径要点

- 入门：跑通 Matmul，建立范式。
- 中阶：实现Matmul算子各个数据通路及同步
- 高阶：Matmul Case 0-8性能优化与理论计算
- 每层映射到样例目录和手册章节。

结论：同一套矩阵编程范式，基础入门→实现学习→性能优化，逐级打磨，开发高性能 Matirx 算子。

02 矩阵算子编程实践：实现一个最基础的Matmul算子



- 1** Host 侧负责资源初始化、内存分配、H2D、kernel 下发、同步、D2H 与落盘



- 2** Device 侧真正执行 AI Core 上的矩阵乘计算



- 3** 核心调用：
mmad_custom <<<<numBlocks, 0, stream>>>
(aDevice, bDevice, cDevice)

02 矩阵算子编程实践：实现一个最基础的Matmul算子



1 核函数 `__global__ __cube__` 运行在Cube核上



2 分核 -> 片上内存分配 -> DataCopy / LoadData / Mmad ? Fixpipe



3 同步事件链 MTE2_MTE1 -> MTE1_M -> M_FIX 把当前骨架串起来

1

```
__global__ __cube__ void mmad_custom(__gm__ uint8_t* a, __gm__ uint8_t* b, __gm__ uint8_t* c)
```

```
//1. 分核 + 片上内存分配
```

```
AscendC::GlobalTensor<half> aGM;  
uint32_t mIterIdx = AscendC::GetBlockIdx() % (M / singleCoreM);  
aGM.SetGlobalBuffer((__gm__ half*)a + mIterIdx * singleCoreM * K);  
AscendC::LocalMemAllocator<AscendC::Hardware::L1> l1Allocator;  
AscendC::LocalMemAllocator<AscendC::Hardware::L0A> l0aAllocator;  
AscendC::LocalTensor<half> a1Local = l1Allocator.Alloc<AscendC::TPosition::A1, half>(.....);  
AscendC::LocalTensor<half> a2Local = l0aAllocator.Alloc<AscendC::TPosition::A2, half>(.....);  
.....
```

2

```
//2. GM -> L1 Buffer
```

```
AscendC::DataCopy(a1Local, aGM, AscendC::Nd2NzParams{.....});  
AscendC::SetFlag<AscendC::HardEvent::MTE2_MTE1>(EVENT_ID0);  
AscendC::WaitFlag<AscendC::HardEvent::MTE2_MTE1>(EVENT_ID0);
```

3

```
//3. L1 Buffer -> L0A/B Buffer
```

```
AscendC::LoadData(a2Local[.....], a1Local[.....], AscendC::LoadData2DParams{.....});  
AscendC::SetFlag<AscendC::HardEvent::MTE1_M>(EVENT_ID0);  
AscendC::WaitFlag<AscendC::HardEvent::MTE1_M>(EVENT_ID0);
```

4

```
//4. Cube
```

```
AscendC::Mmad(cLocal, a2Local, b2Local, AscendC::MmadParams{.....});  
AscendC::SetFlag<AscendC::HardEvent::M_FIX>(EVENT_ID0);  
AscendC::WaitFlag<AscendC::HardEvent::M_FIX>(EVENT_ID0);
```

5

```
//5. Fixpipe
```

```
AscendC::Fixpipe(cGM, cLocal, AscendC::FixpipeParamsV220{.....});  
AscendC::PipeBarrier<PIPE_ALL>();
```

DataCopy
(MTE2)

WaitFlag

MTE2_MTE1

LoadData
(MTE1)

WaitFlag

MTE1_M

Mmad
(Cube)

WaitFlag

M_FIX

Fixpipe



样例地址

https://gitcode.com/cann/asc-devkit/tree/master/examples/01_simd_cpp_api/00_introduction/02_matrix/matmul_basic_api

CANN

目录

Part 1 昇腾矩阵计算基础

Part 2 矩阵算子编程实践

2.1 基础编程入门

2.2 核心特性学习

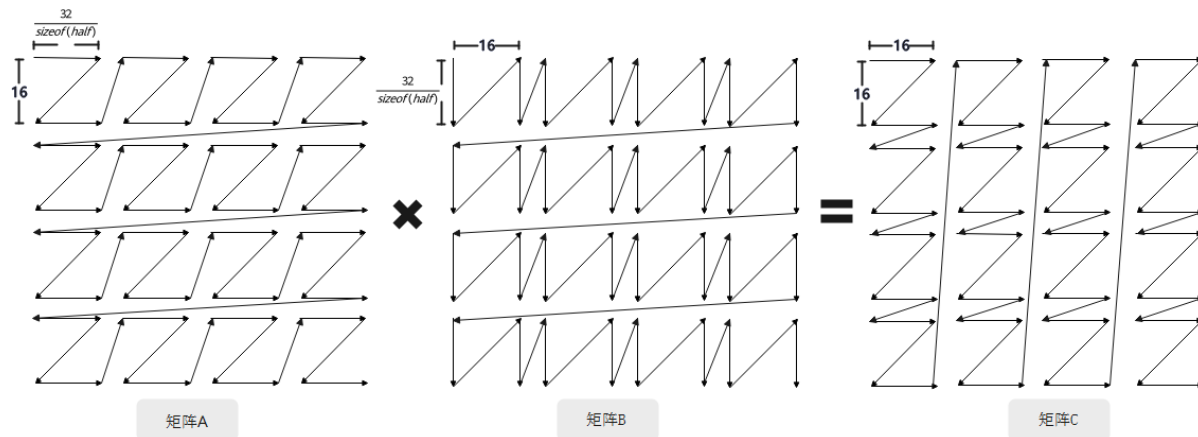
2.3 高性能开发实践

Part 3 能力全景与开发路径

02 矩阵算子编程实践：矩阵乘核心特性学习

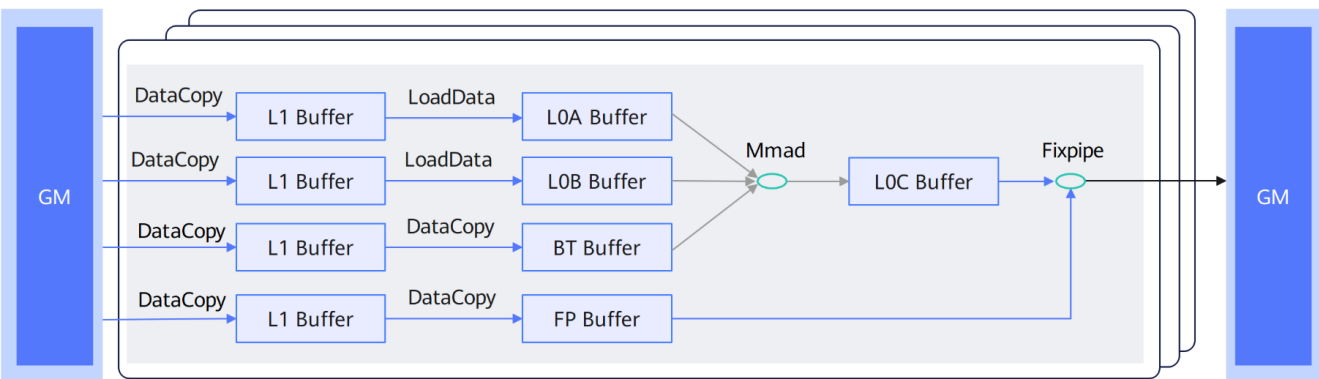
矩阵计算分形介绍：

✦：昇腾AI处理器的矩阵计算单元为追求极高的数据吞吐率与计算密度，采用了专有的分形存储格式。



注：此图针对Atlas A2 训练系列产品/Atlas A2 推理系列产品和Atlas A3训练系列产品/Atlas A3 推理系列产品

矩阵计算流程：



以左矩阵Zz格式为例：

- 定义(What)：大Z(外部行主序) + 小z(内部行主序)
- 物理位置 (Where)：通常位于L0A Buffer，作为左矩阵
- 物理说明 (Why)：
 - 行读取需求：A矩阵是被“横向扫描的”
 - 内存一致性：Zz格式在微观上按行存储，通过GM->L1->L0A搬运需要提前处理好排布；

手册地址

[https://asc.gitcode.com/api/SIMD-API/基础API/矩阵计算 \(ISASI\) /矩阵计算分形介绍/关键分形格式详解.html](https://asc.gitcode.com/api/SIMD-API/基础API/矩阵计算 (ISASI) /矩阵计算分形介绍/关键分形格式详解.html)

- 1、通过Datacopy接口将GM数据搬运到L1 Buffer，并做分型转换；
- 2、通过LoadData接口将数据从L1 Buffer搬入至L0A/B Buffer，分型转换至矩阵乘所需格式；
- 3、通过Mmad接口执行矩阵乘计算；
- 4、通过Fixpipe接口将结果C矩阵搬出

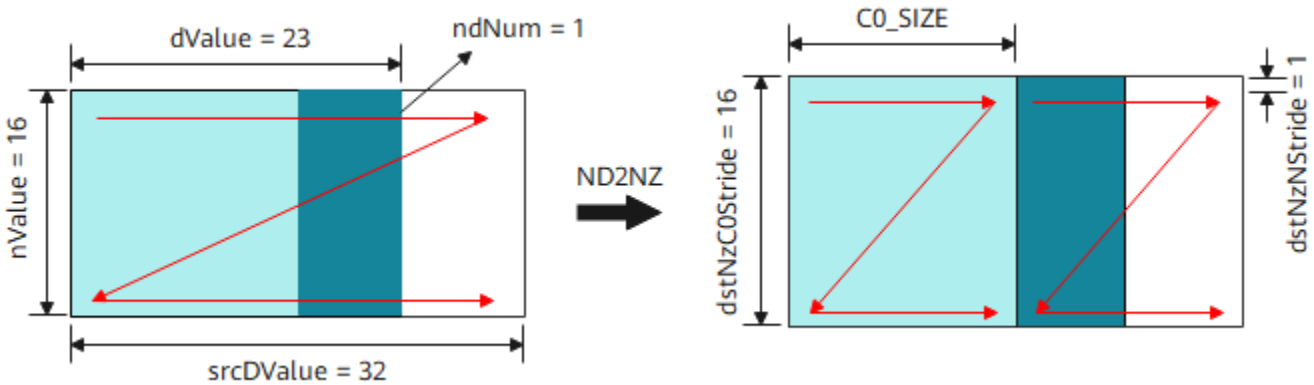
手册地址

[https://asc.gitcode.com/api/SIMD-API/基础API/矩阵计算 \(ISASI\) /概述/矩阵计算流程.html](https://asc.gitcode.com/api/SIMD-API/基础API/矩阵计算 (ISASI) /概述/矩阵计算流程.html)

02 矩阵算子编程实践：矩阵乘核心特性学习

矩阵数据搬入至L1 Buffer:

```
// A矩阵GM->L1调用DataCopy接口 + Nd2NzParams结构体参数:
// {ndNum, nValue, dValue, srcNdMatrixStride, srcDValue, dstNzC0Stride, dstNzNStride, dstNzMatrixStride}
AscendC::DataCopy(a1Local, aGM, AscendC::Nd2NzParams{1, baseM, baseK, 0, K, baseM, 1, 0});
// B矩阵GM->L1调用DataCopy接口 + Nd2NzParams结构体参数:
// {ndNum, nValue, dValue, srcNdMatrixStride, srcDValue, dstNzC0Stride, dstNzNStride, dstNzMatrixStride}
AscendC::DataCopy(b1Local, bGM, AscendC::Nd2NzParams{1, baseK, baseN, 0, N, baseK, 1, 0});
```



手册地址

[https://asc.gitcode.com/api/SIMD-API/基础API/矩阵计算 \(ISASI\) /矩阵计算的搬入/矩阵数据搬入至L1-Buffer/矩阵数据搬入至L1-Buffer.html](https://asc.gitcode.com/api/SIMD-API/基础API/矩阵计算 (ISASI) /矩阵计算的搬入/矩阵数据搬入至L1-Buffer/矩阵数据搬入至L1-Buffer.html)

样例地址

https://gitcode.com/cann/asc-devkit/tree/master/examples/01_simd_cpp_api/03_basic_api/00_data_movement/data_copy_gm2l1

2.2.4.3.4. 矩阵数据搬入至L1-Buffer

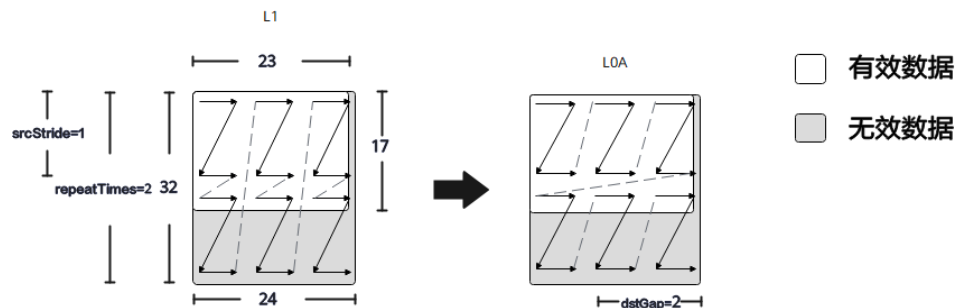
- 2.2.4.3.4.1. GMTOL1-2D矩阵搬运 (LoadData)
- 2.2.4.3.4.2. GMTOL1-2D矩阵搬运V2 (LoadData)
- 2.2.4.3.4.3. GMTOL1连续数据搬运 (DataCopy)
- 2.2.4.3.4.4. GMTOL1高维切分数据搬运 (DataCopy)
- 2.2.4.3.4.5. GMTOL1非对齐数据搬运 (DataCopyPad)
- 2.2.4.3.4.6. GMTOL1随路转换-DN2NZ搬运 (DataCopy)
- 2.2.4.3.4.7. GMTOL1随路转换-ND2NZ搬运 (DataCopy)

scenarioNum	输入格式	输入数据类型	输出数据类型	是否使能Bias	是否使能Vector量化
1	Nz	half	float	否	否
2	ND	half	float	否	否
3	DN	half	float	否	否
4	ND	half	float	是	否
5	ND	half	int8_t	否	是

02 矩阵算子编程实践：矩阵乘核心特性学习

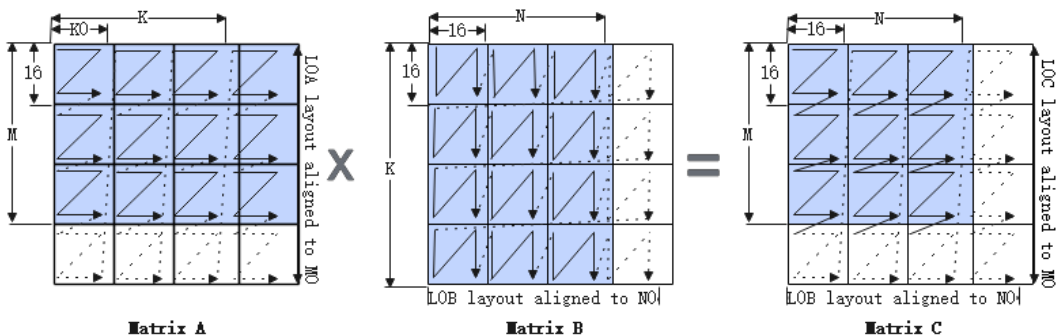
矩阵数据搬入至L0 Buffer:

```
// A矩阵L1->L0A (Nz->Zz) A矩阵L1->L0A调用LoadData接口 + LoadData2DParams结构体参数:  
// {startIndex, repeatTimes, srcStride, sid, dstGap, ifTranspose, addrMode}  
for (int i = 0; i < baseM / CUBE_BLOCK; ++i) {  
    AscendC::LoadData(  
        a2Local[i * baseK * CUBE_BLOCK], a1Local[i * 512 / sizeof(half)],  
        AscendC::LoadData2DParams{0, baseK / CUBE_BLOCK, baseM / CUBE_BLOCK, 0, 0, false, 0});  
}
```



Mmad 计算:

```
// 矩阵乘调用Mmad接口 + MmadParams结构体参数: {m, n, k, unitFlag, cmatrixSource, cmatrixInitVal}  
AscendC::Mmad(cLocal, a2Local, b2Local, AscendC::MmadParams{baseM, baseN, baseK, 0, false, true});
```



手册地址

[https://asc.gitcode.com/api/SIMD-API/基础API/矩阵计算 \(ISASI\) /矩阵计算的搬入/矩阵数据搬入至L0-Buffer/矩阵数据搬入至L0-Buffer.html](https://asc.gitcode.com/api/SIMD-API/基础API/矩阵计算 (ISASI) /矩阵计算的搬入/矩阵数据搬入至L0-Buffer/矩阵数据搬入至L0-Buffer.html)

样例地址

https://gitcode.com/cann/asc-devkit/tree/master/examples/01_simd_cpp_api/03_basic_api/03_matrix_compute/load_data_l12l0

手册地址

[https://asc.gitcode.com/api/SIMD-API/基础API/矩阵计算 \(ISASI\) /Mmad计算/Mmad计算.html](https://asc.gitcode.com/api/SIMD-API/基础API/矩阵计算 (ISASI) /Mmad计算/Mmad计算.html)

样例地址

https://gitcode.com/cann/asc-devkit/tree/master/examples/01_simd_cpp_api/03_basic_api/03_matrix_compute/mmad

02 矩阵算子编程实践：矩阵乘核心特性学习

矩阵计算的搬出：

```
// L0C->GM搬运调用Fixpipe+FixpipeParamsV220结构体参数:  
// {nSize, mSize, srcStride, dstStride, reluEn, quantPre, deqScalar, ndNum, srcNdStride, dstNdStride, unitFlag}  
AscendC::Fixpipe(  
    cGM, cLocal, AscendC::FixpipeParamsV220{baseN, baseM, baseM, N, false, QuantMode_t::F322F16, 0, 1, 0, 0, 0});
```

手册地址

[https://asc.gitcode.com/api/SIMD-API/基础API/矩阵计算 \(ISASI\) /矩阵计算的搬入/矩阵数据搬入至L0-Buffer/矩阵数据搬入至L0-Buffer.html](https://asc.gitcode.com/api/SIMD-API/基础API/矩阵计算 (ISASI) /矩阵计算的搬入/矩阵数据搬入至L0-Buffer/矩阵数据搬入至L0-Buffer.html)

样例地址

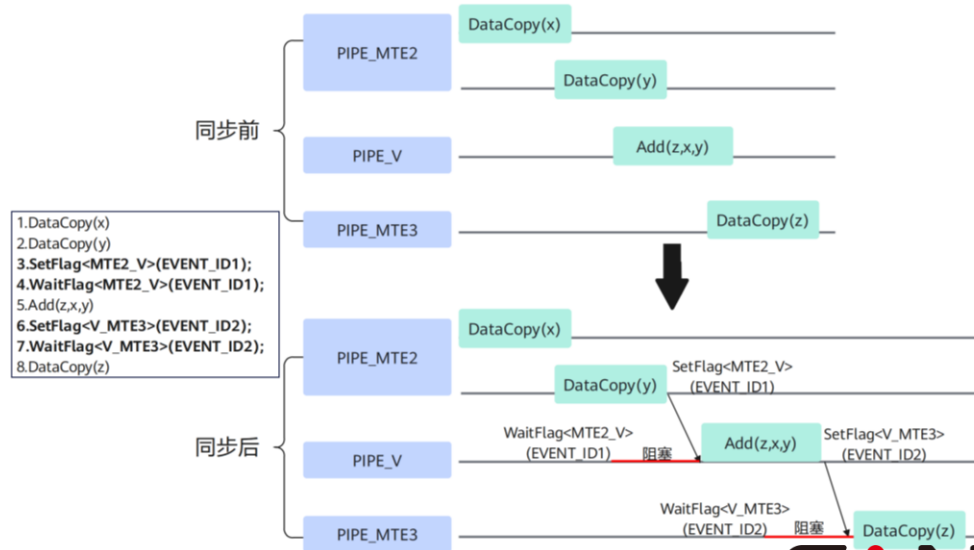
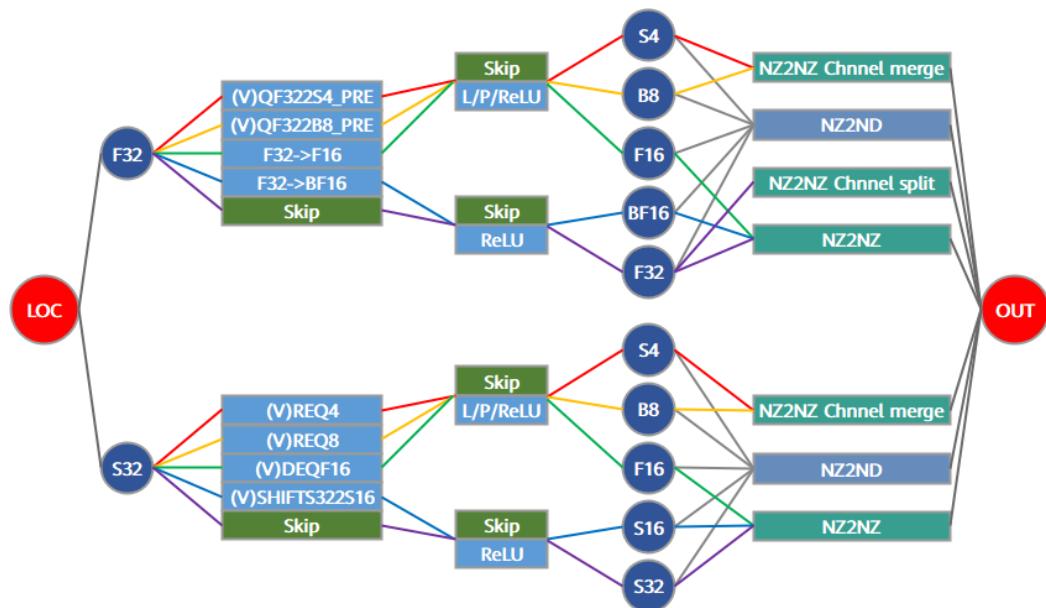
https://gitcode.com/cann/asc-devkit/tree/master/examples/01_simd_cpp_api/03_basic_api/03_matrix_compute/load_data_l1l0

同步控制：

```
// SetFlag用于在生产者流水完成当前任务后写入同步事件, WaitFlag用于让消费者流水等待该事件;  
// HardEvent模板参数描述同步方向, EVENT_ID用于区分同类同步事件。  
// GM->L1的DataCopy由MTE2流水完成, 后续L1->L0的LoadData由MTE1流水读取L1数据;  
// 这里设置MTE2_MTE1事件, 使MTE1等待MTE2, 避免LoadData读取到尚未搬运完成的数据。  
AscendC::SetFlag<AscendC::HardEvent::MTE2_MTE1>(EVENT_ID0);  
AscendC::WaitFlag<AscendC::HardEvent::MTE2_MTE1>(EVENT_ID0);
```

手册地址

[https://asc.gitcode.com/api/SIMD-API/基础API/同步控制/核内同步/SetFlag-WaitFlag\(ISASI\).html](https://asc.gitcode.com/api/SIMD-API/基础API/同步控制/核内同步/SetFlag-WaitFlag(ISASI).html)



目录

Part 1 昇腾矩阵计算基础

Part 2 矩阵算子编程实践

2.1 基础编程入门

2.2 核心特性学习

2.3 高性能开发实践

Part 3 能力全景与开发路径

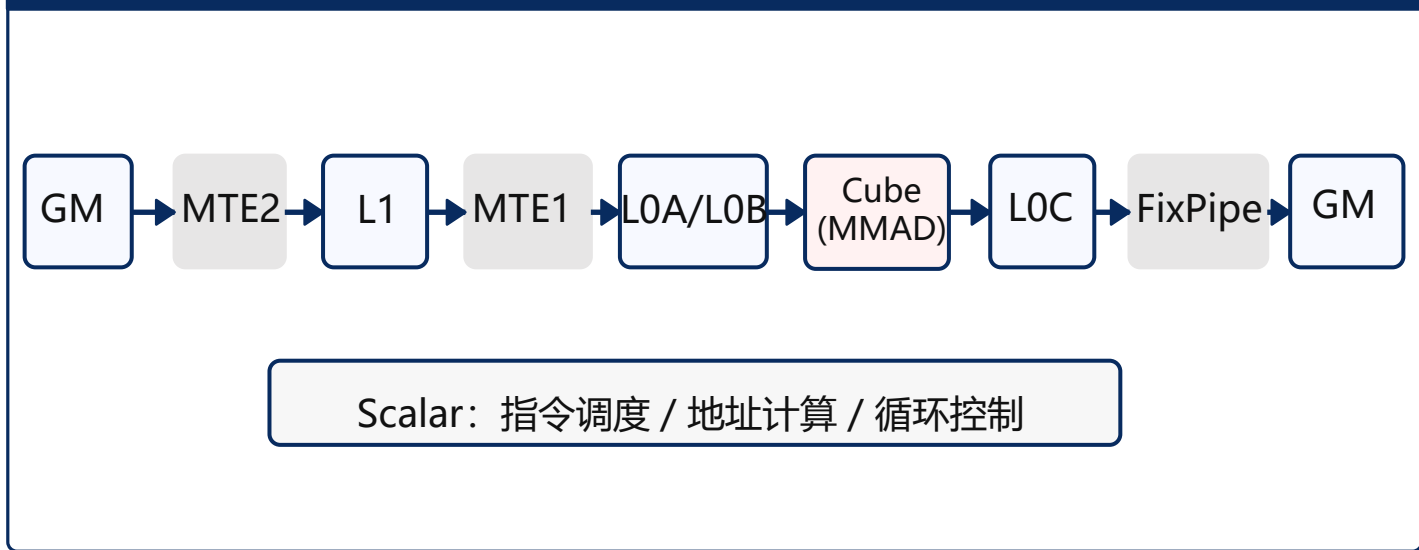
02 矩阵算子编程实践：性能分析

- 用 msprof 采集，首先看 Task Duration 作为端到端总耗时
- 每个指标对应一条硬件流水或调度单元，关注各流水的 time 与 ratio
- 高性能 Matmul 的核心观察指标是 aic_mac_ratio 接近 1

msprof 指标与含义

指标名	对应通路	含义
Task Duration	端到端执行	算子执行总耗时
aic_mte2_time/ratio	Gm->L1	MTE2的时间占比，反映GM到L1的数据加载压力
aic_mte1_time/ratio	L1->L0A/B	MTE1的时间占比，反映L1到L0的数据搬运压力
aic_mac_time/ratio	MMAD	Cube计算单元的时间及占比，反映计算单元利用率
aic_fixpipe_time/ratio	L0C->GM	FixPipe的时间及占比，反映结果写回的访存压力
aic_scalar_time/ratio	Scalar	Scalar标量计算单元的时间及占比

指标对应硬件流水

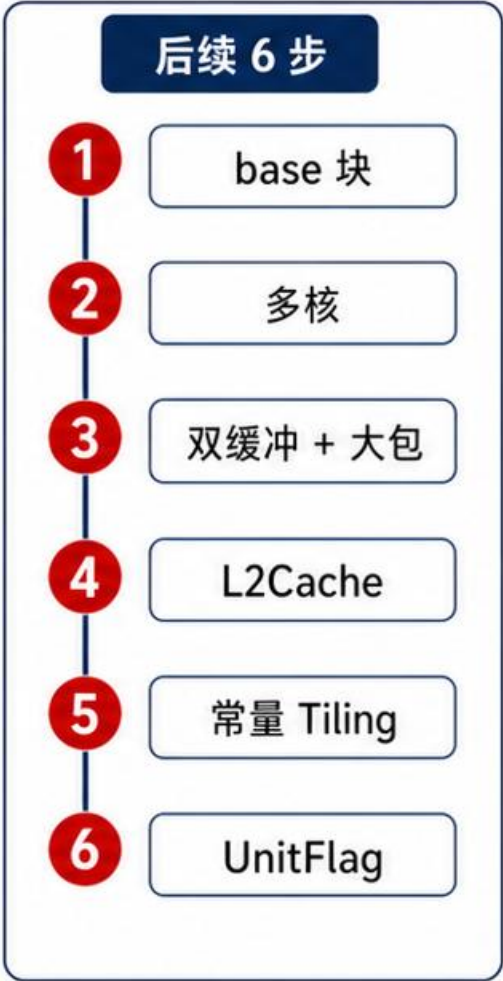


02 矩阵算子编程实践：优化手段与优化结果

M=N=K=8192 Matmul性能阶梯数据

Case	优化手段	核数	Task Duration(us)	aic_mac_ratio	相对Case0优化
0	单核基线	1	759363.98	18.7%	1x
1	base块优化	1	249467.08	32.7%	3.04x
2	多核2x12	24	12541.22	27.6%	60.55x
3	多核4x6	24	12283.84	31.2%	61.82x
4	MDL模板	24	5039.86	69.4%	150.67x
5	L1Cache	24	4156.76	85.4%	182.68x
6	L2Cache	24	4088.36	86.0%	185.74x
7	常量Tiling	24	4053.44	86.3%	187.34x
8	Unitflag	24	4012.44	86.4%	189.25x

注：此数据基于Atlas A2 训练系列产品/Atlas A2 推理系列产品和Atlas A3训练系列产品/Atlas A3 推理系列产品



</> 样例地址

高阶API实现（调优路径）：https://gitcode.com/cann/asc-devkit/tree/master/examples/01_simd_cpp_api/05_best_practices/01_matrix_compute/matmul_high_performance

基础API实现（实现细节）：https://gitcode.com/cann/asc-devkit/tree/master/examples/01_simd_cpp_api/05_best_practices/01_matrix_compute/matmul_basic_api_high_performance

02 矩阵算子编程实践：① base 块 Tiling 优化

➤ **访存计算比：** 每个 cycle 的 Cube 计算需要喂多少字节的数据

Base 块大小为[64, 64, 64]

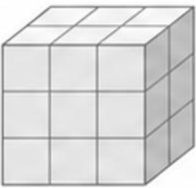
- 计算一个base块所需的cycle为： $64 \times 64 \times 64 / (16 \times 16 \times 16) = 64$ cycles
- 从 L1 Buffer 读： $64 \times 64 \times 2(\text{Byte}) + 64 \times 64 \times 2(\text{Byte}) = 16384(\text{Byte})$
- 访存计算比 = $16384 / 64 = 256 \text{ B/cycle}$

Base 块大小为[128, 256, 64]

- 计算一个base块所需的cycle为： $128 \times 256 \times 64 / (16 \times 16 \times 16) = 512$ cycles
- 从 L1 Buffer 读： $128 \times 64 \times 2(\text{Byte}) + 256 \times 64 \times 2(\text{Byte}) = 49152(\text{Byte})$
- 访存计算比 = $49152 / 512 = 96 \text{ B/cycle}$

小 base 块 (Case 0)

[64, 64, 64]



单位计算需要搬的数据多

调大 base 块



单位计算需要搬的数据减少



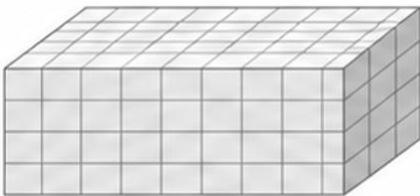
带宽压力下降



计算效率提升

大 base 块 (Case 1)

[128, 256, 64]



单位计算需要搬的数据更少

性能数据：🔴 表格中耗时单位均为μs。

	Task Duration	aic_mac_time	aic_mac_ratio	aic_scalar_time	aiv_scalar_ratio	aic_mte1_time	aic_mte1_ratio	aic_mte2_time	aic_mte2_ratio	aic_fixpipe_time	aic_fixpipe_ratio
Case0	759363.98	141699.46	0.187	508787.51	0.67	162104.18	0.213	582515.83	0.767	34303.06	0.045
Case1	249467.08	81477.19	0.327	63608.82	0.255	52003.70	0.208	195613.43	0.784	11313.75	0.045
	3.04x							2.98x			

02 矩阵算子编程实践： ① base 块 Tiling 优化



约束与提示：

- Base块大小不是越大越好，需要满足L0A/L0B Buffer、L0C Buffer容量大小

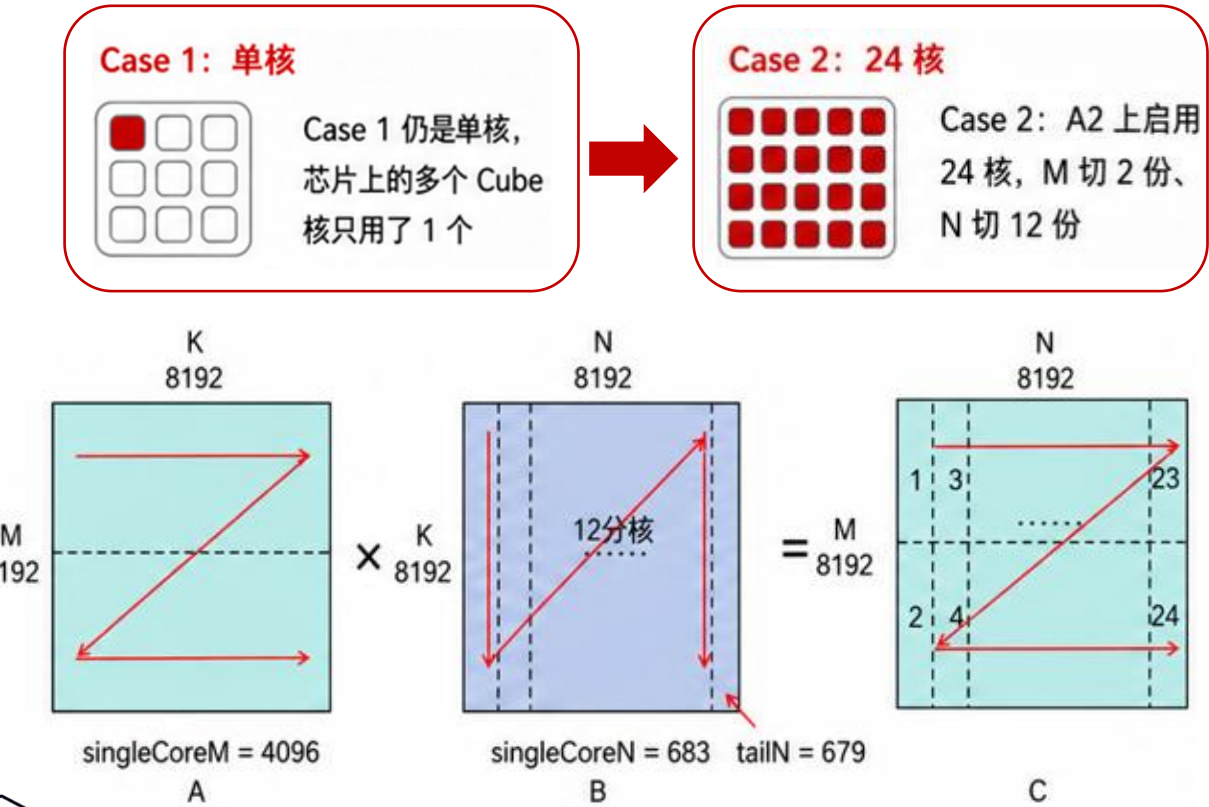
产品	Atlas A2/A3	Ascend 950PR/DT
L0A/B Buffer容量	64KB	64KB
L0C Buffer容量	128KB	256KB
[baseM, baseN, baseK]	输入b16数据类型：[128,256,64]	输入b16数据类型：[256,256,64]
推荐Tiling设置	输入b8类型：[128,256,128]	输入b8类型：[256,256,128]



L0A/B Buffer当前分别只有16KB和32KB->为后续的**Double Buffer**做准备。

02 矩阵算子编程实践： ②采用多核切分

➤ 多核切分： 启用多个aic核并行计算



基础API中多核切分实现示例

```
// 1.核函数指定使用核数numBlocks
mmad_custom<.....><<<numBlocks, nullptr, stream>>>(aDevice, bDevice,
cDevice);
// 2. 计算当前核在 M/N 方向的迭代索引, 确定 GM 起始偏移
constexpr uint32_t mIter = DivCeil(M, singleCoreM);
uint32_t mIterIdx = AscendC::GetBlockIdx() % mIter;
uint32_t nIterIdx = AscendC::GetBlockIdx() / mIter;

uint64_t gmOffsetA = mIterIdx * singleCoreM * K;
uint64_t gmOffsetB = nIterIdx * K * singleCoreN;
uint64_t gmOffsetC = mIterIdx * singleCoreM * N + nIterIdx *
singleCoreN;
aGM = aGMOri[gmOffsetA];
bGM = bGMOri[gmOffsetB];
cGM = cGMOri[gmOffsetC];

// 3. 计算当前核实际的 M/N 维度大小 (最后一块可能不满 singleCore
actualSingleCoreM = M - mIterIdx * singleCoreM;
actualSingleCoreM = actualSingleCoreM < singleCoreM ?
actualSingleCoreM : singleCoreM;
actualSingleCoreN = N - nIterIdx * singleCoreN;
actualSingleCoreN = actualSingleCoreN < singleCoreN ?
actualSingleCoreN : singleCoreN;
```

性能数据： 表格中耗时单位均为μs。

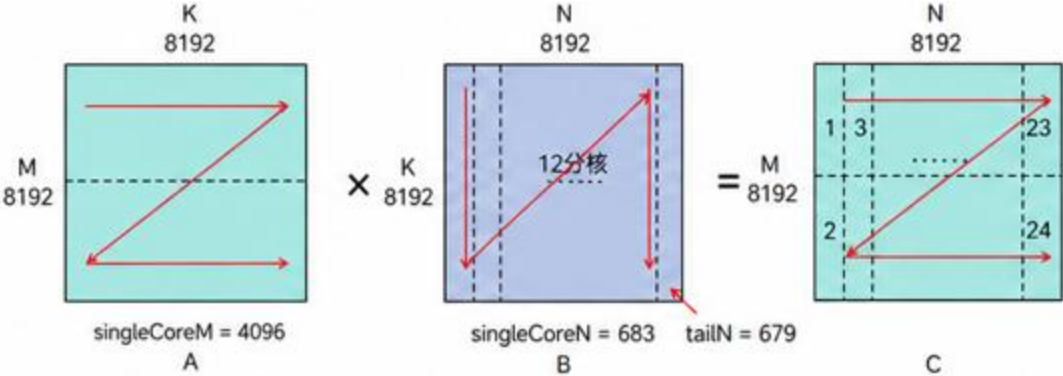
	Task	Duration	aic_mac_time	aic_mac_ratio	aic_scalar_time	aiv_scalar_ratio	aic_mte1_time	aic_mte1_ratio	aic_mte2_time	aic_mte2_ratio	aic_fixpipe_time	aic_fixpipe_ratio
Case1		249467.08	81477.19	0.327	63608.82	0.255	52003.70	0.208	195613.43	0.784	11313.75	0.045
Case2		12541.22	3462.802	0.276	2987.47	0.238	2260.551	0.18	10177.798	0.812	875.439	0.07

19.89x

02 矩阵算子编程实践： ②采用多核切分

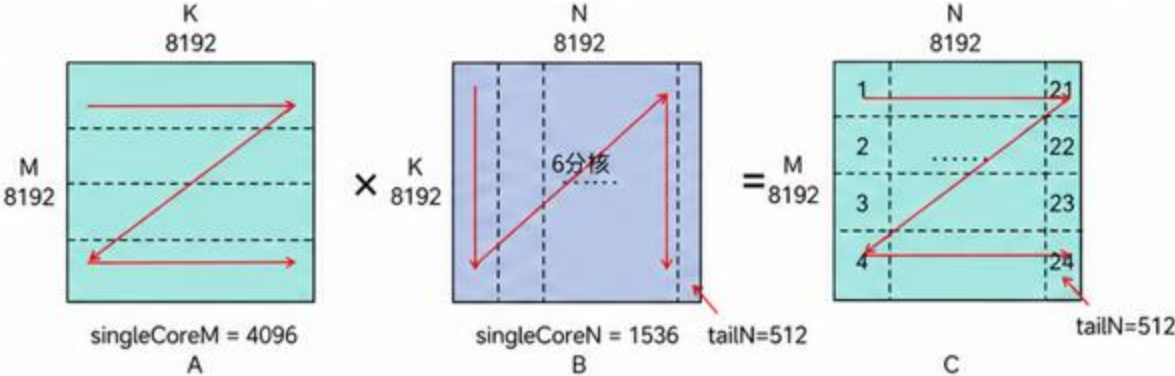
- 同样 24 核，切法不同、性能不同
- Case3 改为4×6分核，SingleShape为(2048,1536,8192)
- 4×6的 singleN=1536，尾块tailN=512；满足512字节对齐；
- 同地址访问数从12核抢同一块数据降到6核，串行延迟更低

Case2: 2 × 12 分核



- ❌ singleN=683 未对齐 (非 512B 整数倍)
- ❌ 12 核同时访问同一行 (同地址访问数=12)

Case3: 4 × 6 分核



- ✅ singleN=1536=3×512 对齐 (512B 整数倍)
- ✅ 6 核访问同一行 (同地址访问数=6), 串行延迟更低

性能数据：✦表格中耗时单位均为μs。

	Task Duration	aic_mac_time	aic_mac_ratio	aic_scalar_time	aiv_scalar_ratio	aic_mte1_time	aic_mte1_ratio	aic_mte2_time	aic_mte2_ratio	aic_fixpipe_time	aic_fixpipe_ratio
Case2	12541.22	3462.802	0.276	2987.47	0.238	2260.551	0.18	10177.798	0.812	875.439	0.07
Case3	12283.84	3394.884	0.312	2656.33	0.244	2166.824	0.199	8616.671	0.793	488.829	0.045

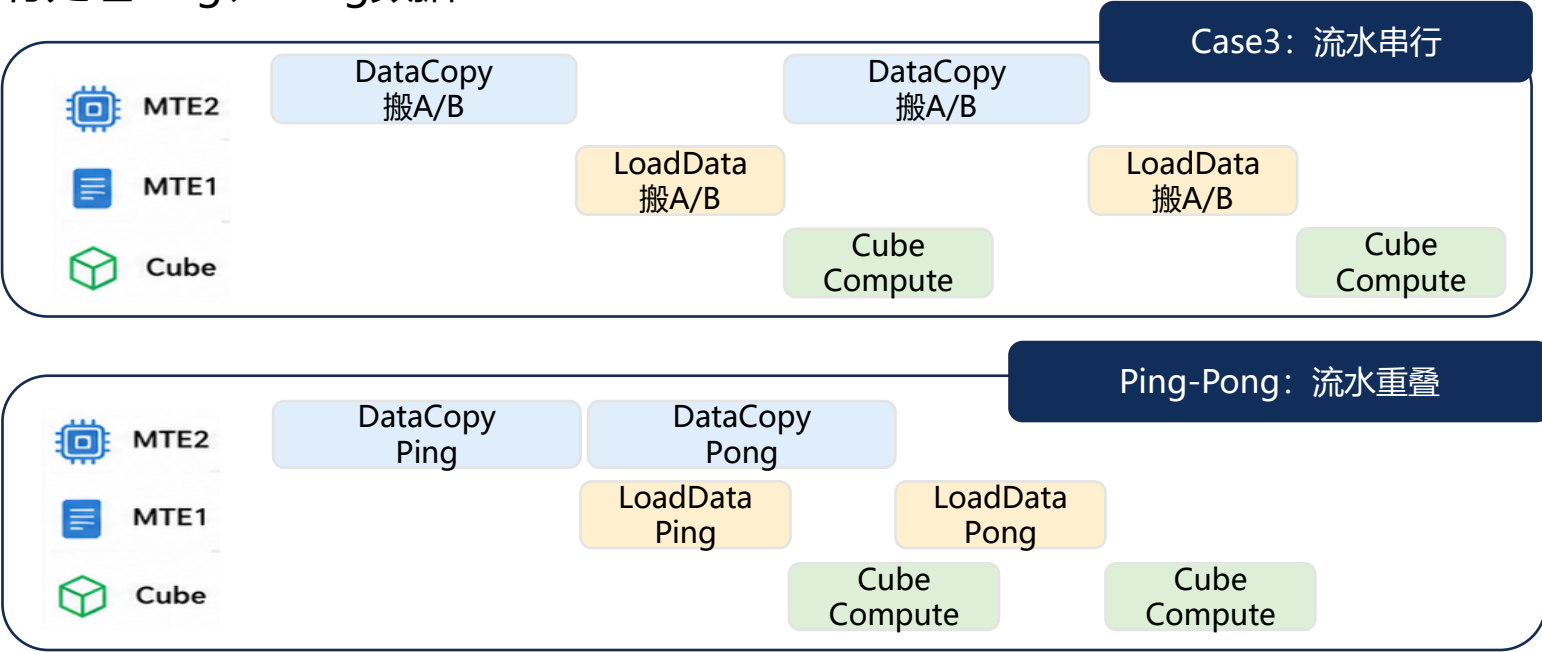
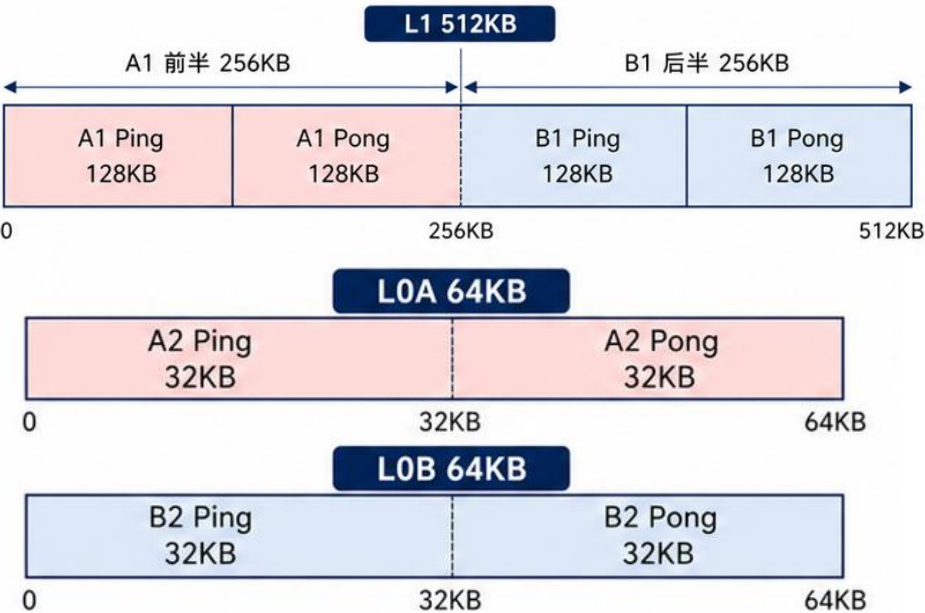
15.33%

</> 样例地址

DataCopy最佳实践: https://gitcode.com/cann/asc-devkit/tree/master/examples/01_simd_cpp_api/05_best_practices/04_memory_access/data_copy

02 矩阵算子编程实践：③双缓冲 + 大包搬运

- Case3: Cube利用率仅31.2%，且各流水均未达到bound状态
- Double Buffer: 将待处理的数据一份为二，并行处理Ping、Pong数据



性能数据: 表格中耗时单位均为μs。

	Task Duration	aic_mac_time	aic_mac_ratio	aic_scalar_time	aiv_scalar_ratio	aic_mte1_time	aic_mte1_ratio	aic_mte2_time	aic_mte2_ratio	aic_fixpipe_time	aic_fixpipe_ratio
Case3	12283.84	3394.884	0.312	2656.33	0.244	2166.824	0.199	8616.671	0.793	488.829	0.045
仅开db	9621.85	3394.88	0.399	3857.561	0.453	2327.96	0.273	8501.544	0.999	46.055	0.005



手册地址

<https://asc.gitcode.com/guide/技术附录/概念原理和术语/性能优化技术原理/DoubleBuffer.html>

MTE2 bound

02 矩阵算子编程实践：③双缓冲 + 大包搬运

➤ **大包搬运：** MTE2一次性搬运多个基本块，提高带宽利用率



1 输入矩阵shape较大，但每次只搬运一个基本块

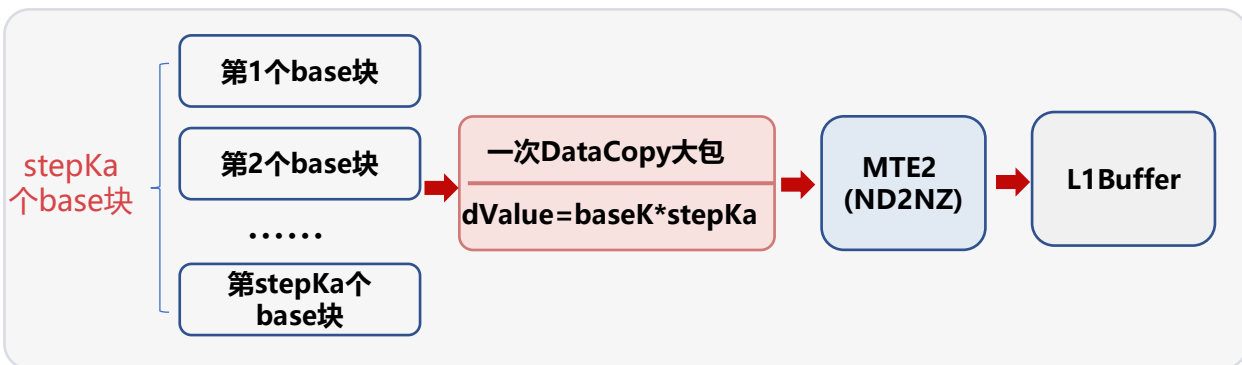


2 频繁调用MTE2搬运指令，MTE2带宽利用率低

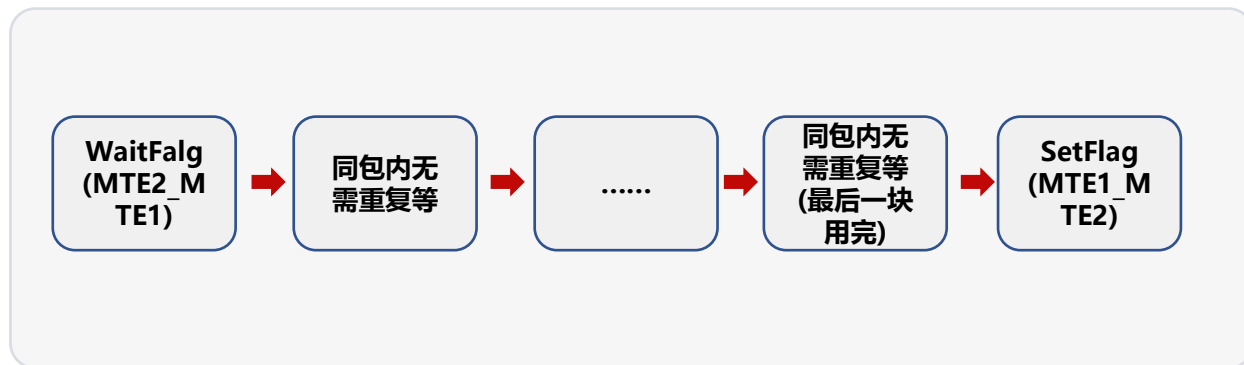


3 结果：MTE2耗时长，阻塞后续的MTE1和MMAD流水，瓶颈从Cube空等转移到搬运

把多个 base 块打包成一次DataCopy(大包):



MTE1 复用L1 Buffer上的数据



	Task Duration	aic_mac_time	aic_mac_ratio	aic_scalar_time	aiv_scalar_ratio	aic_mte1_time	aic_mte1_ratio	aic_mte2_time	aic_mte2_ratio	aic_fixpipe_time	aic_fixpipe_ratio
--	---------------	--------------	---------------	-----------------	------------------	---------------	----------------	---------------	----------------	------------------	-------------------

仅开db	9621.85	3394.88	0.399	3857.561	0.453	2327.96	0.273	8501.544	0.999	46.055	0.005
Case5	4156.76	3352.087	0.854	1464.492	0.373	2750.454	0.701	3778.555	0.963	47.853	0.012

56.8%

55.6%



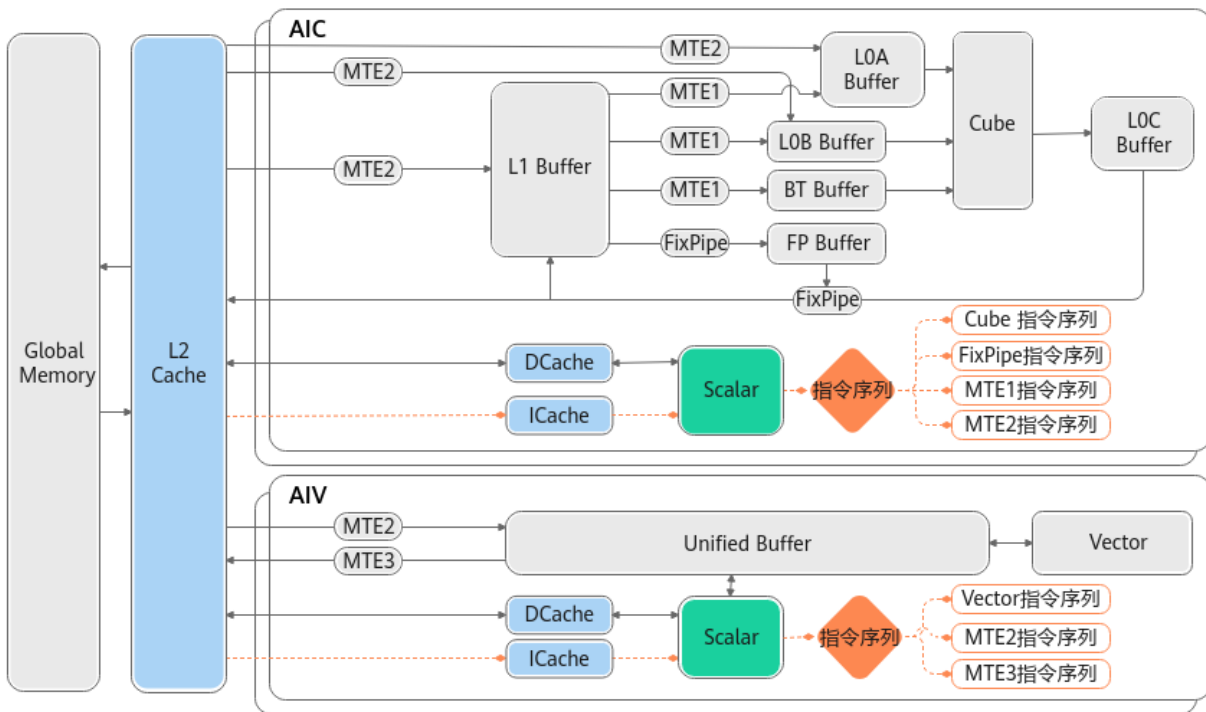
手册地址

<https://asc.gitcode.com/guide/算子实践参考/优秀实践/Matmul性能调优案例/Matmul高阶API使能MDL模板.html>

CANN

02 矩阵算子编程实践：④L2 Cache 切分

➤ **L2Cache切分：** L2Cache命中可提高访问速度， 适量切分可提高缓存命中率



切分策略：A 沿 M轴切两份，B驻留L2 复用

$$A + B + C = 384\text{MB} > \text{L2 } 192\text{MB}$$



- 第 1 轮： $C1 = A1 \times B$ ，B从 GM 载入并驻留L2
- 第 2 轮： $C2 = A2 \times B$ ，B 可以复用L2内数据

	Task Duration	aic_mac_time	aic_mac_ratio	aic_scalar_time	aiv_scalar_ratio	aic_mte1_time	aic_mte1_ratio	aic_mte2_time	aic_mte2_ratio	aic_fixpipe_time	aic_fixpipe_ratio
Case5	4156.76	3352.087	0.854	1464.492	0.373	2750.454	0.701	3778.555	0.963	47.853	0.012
Case6	4088.36	3254.31	0.86	1753.463	0.463	2680.849	0.708	3625.42	0.958	47.398	0.013



手册地址

<https://asc.gitcode.com/api/SIMD-API/基础API/缓存控制/系统缓存概述.html>

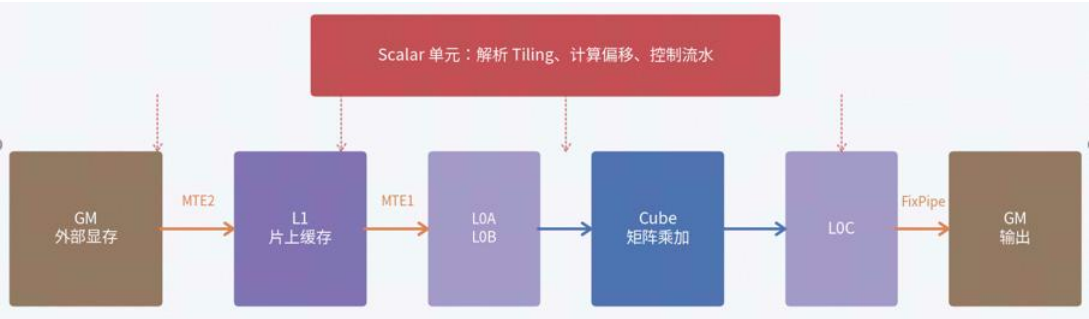
<https://asc.gitcode.com/guide/算子实践参考/SIMD算子性能优化/内存访问/设置合理的L2-CacheMode.html>

CANN

02 矩阵算子编程实践：⑤Tiling 常量化的

➤ **Tiling 常量化**：将Scalar计算提前到编译期进行，以减少运行时的Scalar计算开销

Scalar影响指令头开销与流水阻塞：



编译期常量折叠的基本原理：



	Task Duration	aic_mac_time	aic_mac_ratio	aic_scalar_time	aiv_scalar_ratio	aic_mte1_time	aic_mte1_ratio	aic_mte2_time	aic_mte2_ratio	aic_fixpipe_time	aic_fixpipe_ratio
Case6	4088.36	3254.31	0.86	1753.463	0.463	2680.849	0.708	3625.42	0.958	47.398	0.013
Case7	4053.44	3163.682	0.863	968.616	0.264	2609.617	0.712	3513.806	0.959	45.76	0.012

45%

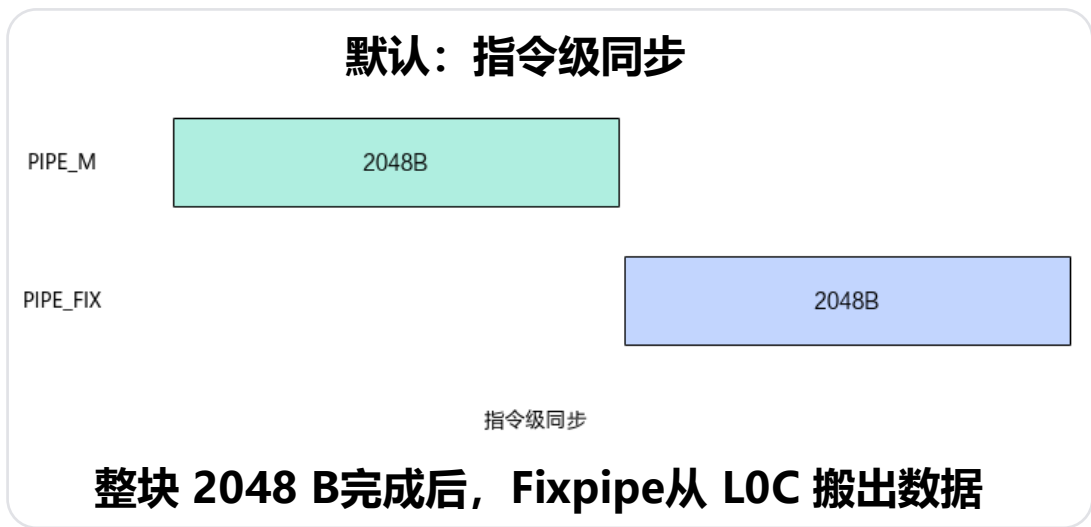
手册地址

<https://asc.gitcode.com/guide/算子实践参考/优秀实践/Matmul性能调优案例/Matmul高阶API使能Tiling全量常量化.html>

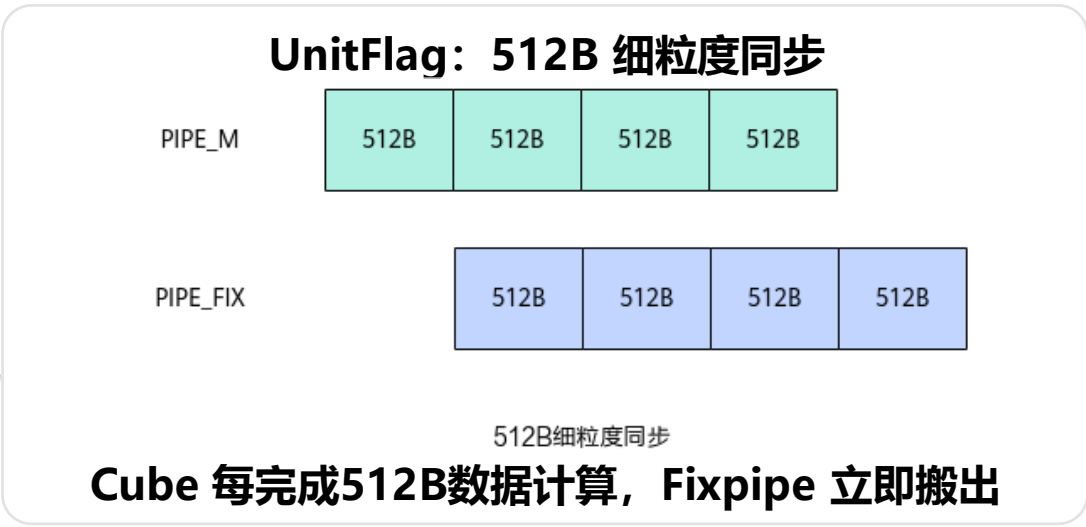
CANN

02 矩阵算子编程实践： ⑥开启unitflag

- **Unitflag机制：**UnitFlag为MMAD计算指令和FIXPIPE数据搬运指令提供了**基于内存访问的细粒度同步**，使计算与搬运流水并行。



同步粒度下沉



	Task Duration	aic_mac_time	aic_mac_ratio	aic_scalar_time	aiv_scalar_ratio	aic_mte1_time	aic_mte1_ratio	aic_mte2_time	aic_mte2_ratio	aic_fixpipe_time	aic_fixpipe_ratio
Case7	4053.44	3163.682	0.863	968.616	0.264	2609.617	0.712	3513.806	0.959	45.76	0.012
Case8	4012.44	3076.396	0.864	1026.069	0.288	2533.512	0.712	3435.584	0.965	272.576	0.077



手册地址

<https://asc.gitcode.com/guide/算子实践参考/优秀实践/Matmul性能调优案例/Matmul高阶API使能UnitFlag.html>

CANN

02 矩阵算子编程实践：理论性能对比



Cube 理论时间：

$$cube_time = \frac{M \times N \times K}{16 \times 16 \times 16 \times core_num \times cube_freq}$$



MTE2 理论搬运时间：

$$MTE2\text{理论耗时} = \frac{GM\text{读入数据总量}}{1.8TB/s} + \frac{L2Cache\text{读入数据总量}}{5TB/s}$$

Atlas A2/A3 计算：24核 @1.85GHZ，GM 带宽 1.8TB/s，L2Cache带宽约为5TB/s

Cube:	理论3022.92μs		实测 3076.4μs	误差 1.77%
MTE2:	理论 ~2672μs		实测 3435.6μs	误差 ~28%

Ascend 950 计算：32核 @1.65GHZ，GM 带宽 1.6TB/s，L2Cache带宽约为5TB/s

Cube:	理论 2542μs		实测 2549.7μs	误差 0.3%
MTE2:	理论 ~1832.1μs		实测 1900.3μs	误差 ~3.72%

📁 手册地址

<https://asc.gitcode.com/guide/算子实践参考/性能分析.html>

CANN

目录

Part 1 昇腾矩阵计算基础

Part 2 矩阵算子编程实践

2.1 基础编程入门

2.2 核心特性学习

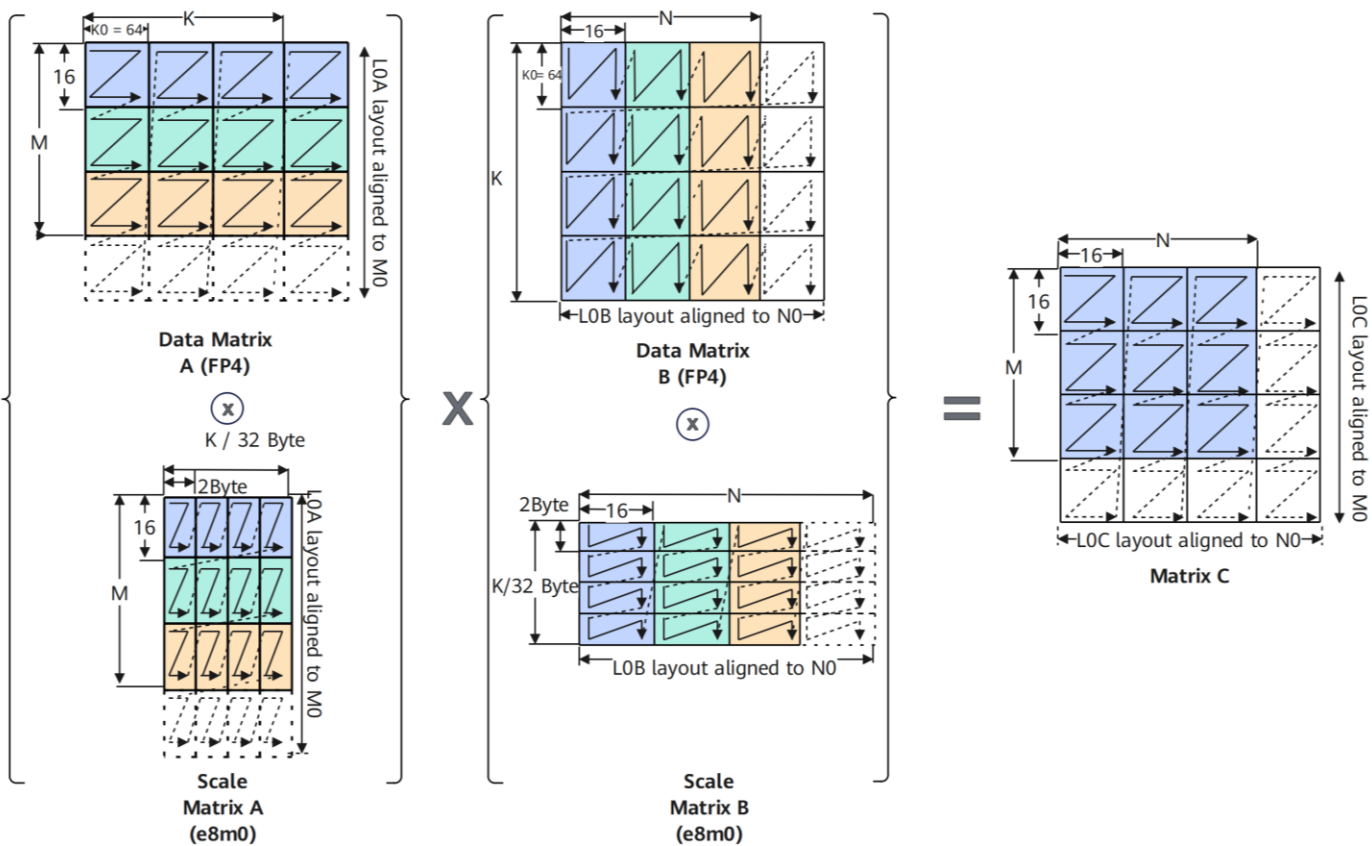
2.3 高性能开发实践

Part 3 能力全景与开发路径

03 能力全景： MxMamul最佳实践

- **MXMatmul (Microscaling)** : Ascend 950系列产品新增功能, 通过“分组共享缩放因子”的方式在减少内存占用的同时, 最大限度地保持计算精度。

$$C = (\text{scaleA} \otimes A) \times (\text{scaleB} \otimes B)$$



手册地址

分形介绍:

[https://asc.gitcode.com/api/SIMD-API/基础API/矩阵计算 \(ISASI\) /矩阵计算分形介绍/辅助矩阵分形格式详解.html](https://asc.gitcode.com/api/SIMD-API/基础API/矩阵计算(ISASI)/矩阵计算分形介绍/辅助矩阵分形格式详解.html)

MXMatmul:

[https://asc.gitcode.com/api/SIMD-API/基础API/矩阵计算 \(ISASI\) /Mmad计算/MmadMx.html](https://asc.gitcode.com/api/SIMD-API/基础API/矩阵计算 (ISASI) /Mmad计算/MmadMx.html)

入门实现:

https://gitcode.com/cann/asc-devkit/tree/master/examples/01_simd_cpp_api/03_basic_api/03_matrix_compute/mmad_mx

高阶API性能实践:

https://gitcode.com/cann/asc-devkit/tree/master/examples/01_simd_cpp_api/05_best_practices/01_matrix_compute/matmul_mxfp4_high_performance

基础API性能实践:

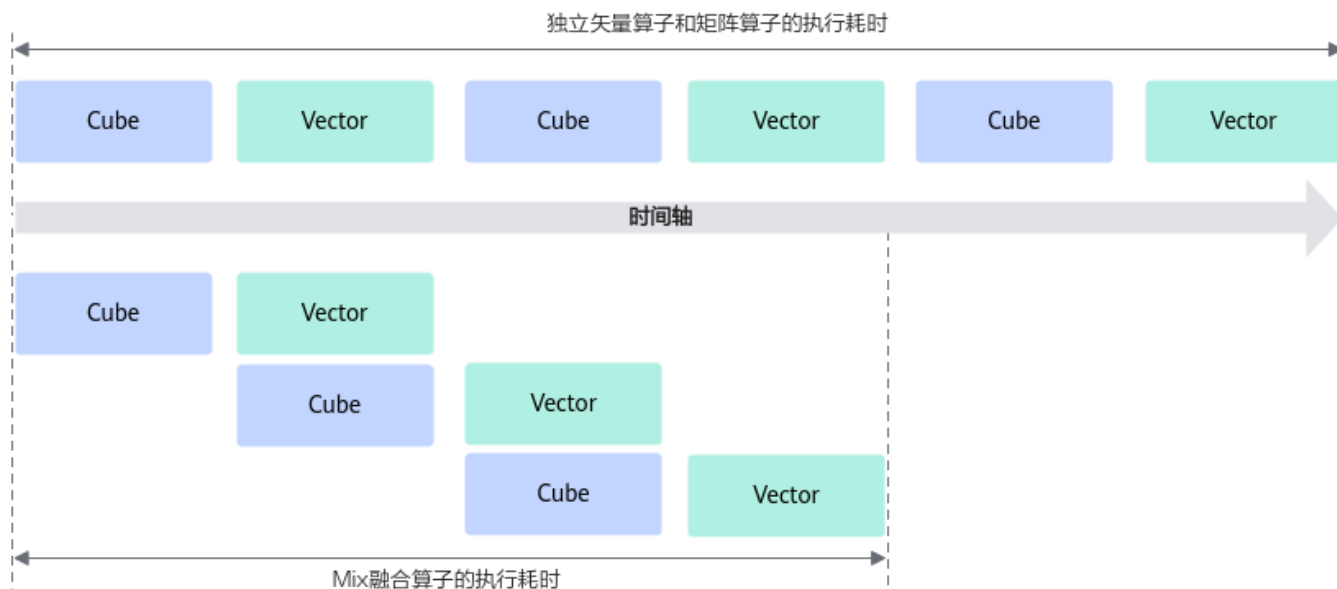
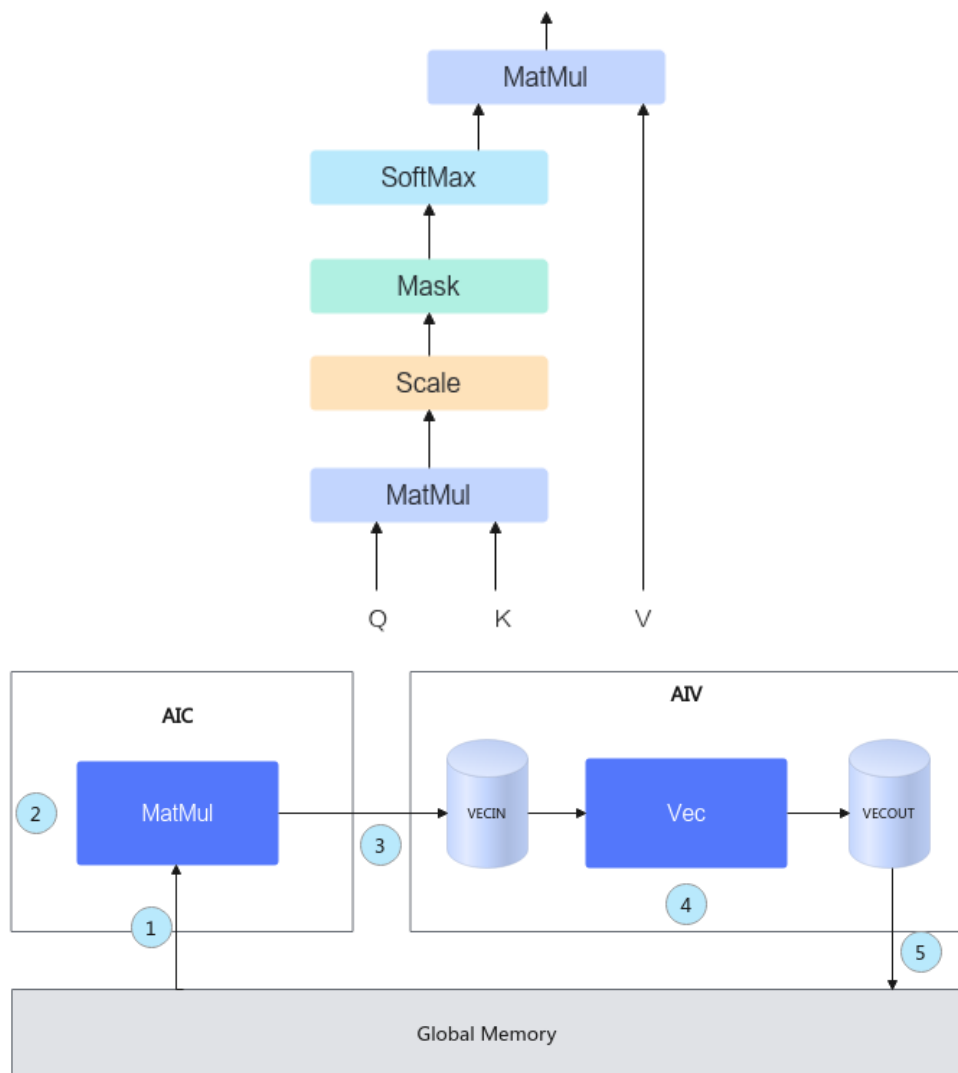
https://gitcode.com/cann/asc-devkit/tree/master/examples/01_simd_cpp_api/05_best_practices/01_matrix_compute/matmul_mxfp4_basic_api_high_performance

样例地址

CANN

03 能力全景：CV融合最佳实践

- 融合了Cube计算、Vector计算的算子统称为**CV融合算子**
- 通过融合算子编程的方式，将矢量算子和矩阵算子进行融合，**并行化Cube侧和Vector侧流水**，获得性能收益



📁 手册地址

<https://asc.gitcode.com/guide/算子实践参考/SIMD算子实现/融合算子编程/融合算子编程.html>

入门实现：

https://gitcode.com/cann/asc-devkit/tree/master/examples/01_simd_cpp_api/00_introduction/03_fusion_operation/matmul_leakyrelu_basic_api

📄 样例地址

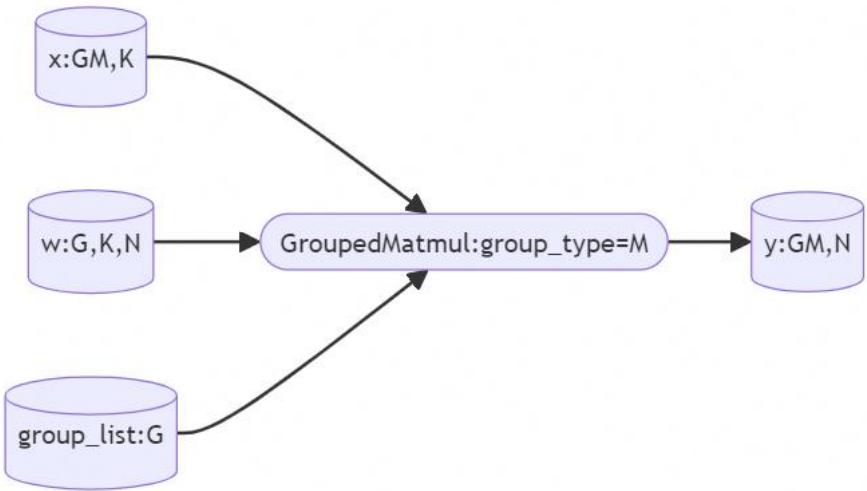
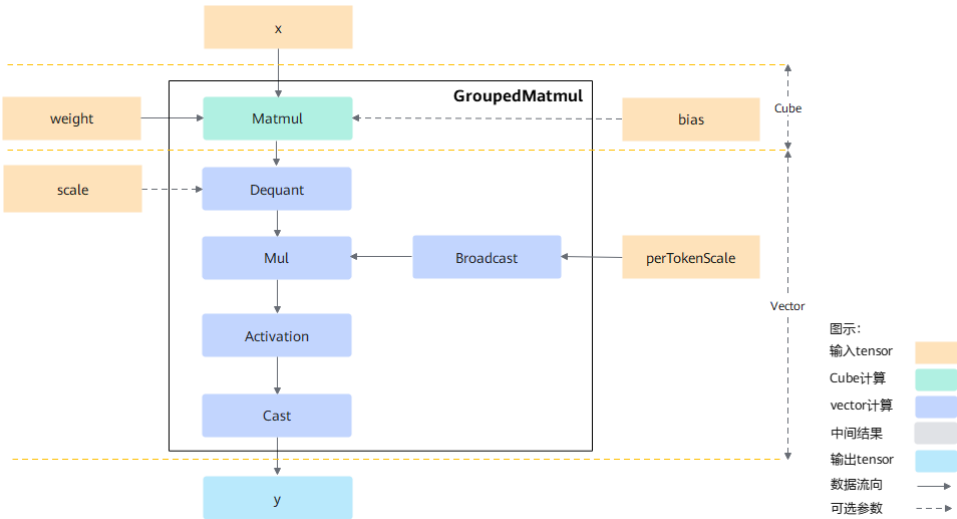
基础API性能实践：

https://gitcode.com/cann/asc-devkit/tree/master/examples/01_simd_cpp_api/05_best_practices/03_fusion_compute/matmul_gelu_high_performance

03 能力全景： GroupedMatmul最佳实践

- GroupedMatmul算子的功能是实现分组矩阵矩阵乘计算。
- 提供最佳实践样例介绍对于Vector计算占比较高场景下的CV融合优化手段

样例类型(OpType)	QuantGroupMatmul			
	name	shape	data type	format
样例输入	x	[8192, 1024]	int8	ND
	weight	[8, 1024, 8192]	int8	NZ
	group	[8]	int64	ND
	scale	[8, 8192]	float	ND
	perTokenScale	[8192]	float	ND
样例输出	y	[8192, 8192]	float16	ND



手册地址

<https://asc.gitcode.com/guide/算子实践参考/优秀实践/GroupedMatmul算子性能调优案例.html>

样例地址

性能实践：

https://gitcode.com/cann/asc-devkit/tree/master/examples/01_simd_cpp_api/05_best_practices/03_fusion_compute/quant_group_matmul_high_performance

03 开发路径 手册 + 样例：你的两个工具箱

 **手册 docs/**
讲原理、流程和 API

 **原理与架构**
AI Core、SIMD、存储层级
(GM / UB / Reg / VF)

 **开发流程**
Host → Device → AI Core
编译、运行与调试

 **API 参考**
Ascend C API 详解
DataCopy、PipeBarrier 等

 **API 速查**
SIMD API 速查手册
(docs/api/SIMD-API/)

目录示例

```
asc-devkit/docs
├── api/
├── guide/
│   ├── 入门教程
│   ├── 编程指南
│   ├── 算子实践参考
│   └── 兼容性迁移指南
└── .....
```

 **样例 examples/**
给可编译、可运行的完整工程

 **完整工程**
可直接编译、运行
快速验证思路

 **典型算子**
Add、Gelu、MatMul 等
覆盖常见场景

 **性能优化样例**
多核、双缓冲、Reg/VF
融合等进阶实践

 **对照学习**
配套 README 与注释
便于理解与对照

目录示例

```
asc-devkit/examples/01_simd_cpp_api
├── 00_introduction
├── 01_utilities
├── 02_features
├── 03_basic_api
├── 04_advanced_api
├── 05_best_practices
├── 06_compatibility_guide
└── .....
```

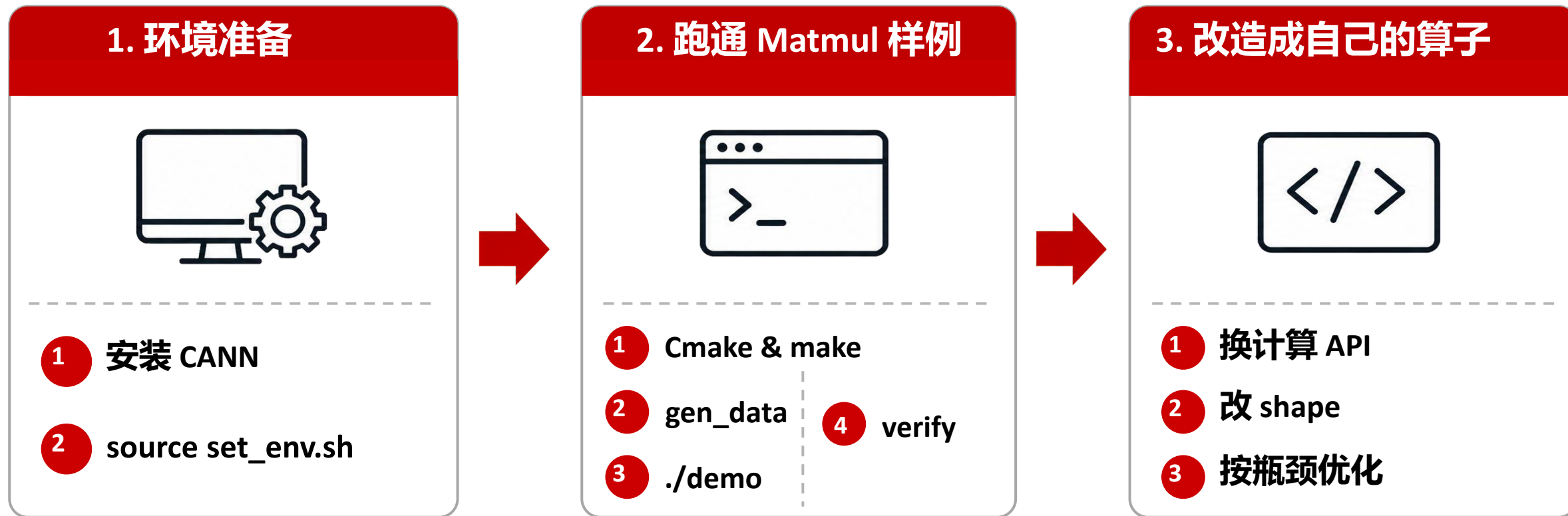
✓ 手册 docs/ 讲原理、流程、API 以及实践指南。

✓ 样例 examples/ 给可编译、可运行的完整工程。

 手册地址: <https://asc.gitcode.com/>

 样例地址: <https://gitcode.com/cann/asc-devkit/tree/master/examples>

03 开发路径 手册 + 样例：你的两个工具箱



结合手册与样例，从能跑开始，再逐步追性能。



手册地址：

<https://asc.gitcode.com>



样例地址：

<https://gitcode.com/cann/asc-devkit/blob/master/examples>

Thank you.



Ascend C 算子编程交流群

<https://gitcode.com/cann>

社区愿景：打造开放易用、技术领先的AI算力新生态

社区使命：使能开发者基于CANN社区自主研究创新，构筑根深叶茂、跨产业协同共享共赢的CANN生态

Vision: Building an Open, Easy-to-Use, and Technology-leading AI Computing Ecosystem

Mission: Enable developers to independently research and innovate based on the CANN community and build a win-win CANN ecosystem with deep roots and cross-industry collaboration and sharing.



上CANN社区获取干货



关注CANN公众号获取资讯

CANN